

PROJECT-1

Simulation of the FHP-1 Lattice Gas Model

Bodige Sai Vineeth Goud (2328268)
Sidhesh Prakash Jadhav (2328016)

January 26, 2025

Abstract

In this report, we describe the implementation and simulation of the FHP-1 lattice gas model for simulating fluid dynamics. The model was implemented in C with MPI parallelization. The simulation runs on grids of various sizes, including obstacles and boundary conditions, and we test its performance with different processor configurations. Results are compared in terms of density and velocity fields, as well as the efficiency of the parallelized implementation.

1 Introduction

The FHP-1 lattice gas model is a discrete lattice-based method for simulating fluid dynamics in two dimensions. The model uses a hexagonal grid to represent fluid cells, and each cell can store information about the direction of fluid particles. The FHP-1 model simulates incompressible fluids, and it has been shown that it can reproduce the behavior of the Navier-Stokes equations.

The system evolves in discrete time steps through a collision and streaming process. The collision phase updates the configuration of each grid point based on the local state, and the streaming phase moves particles to adjacent grid points. The model uses different boundary conditions such as periodic, Dirichlet, no-slip, and slip boundaries.

In this report, we describe the implementation of the FHP-1 model, the methodology used to simulate the system, and the results obtained from running the simulation on various grid sizes and processor configurations.

2 Implementation Details

The simulation is implemented in C using the Message Passing Interface (MPI) for parallelization. The main components of the program include:

- **Grid Initialization:** A 2D grid of size $X \times Y$ is initialized, with a population density representing the probability of particles in each grid cell.
- **Collision Phase:** Each grid point undergoes a collision process where particles are rearranged according to predefined rules.

- **Streaming Phase:** Particles are moved to neighboring grid points based on the direction of movement.
- **Boundary Conditions:** Different boundary conditions (periodic, Dirichlet, no-slip, and slip) are applied based on the configuration.
- **Parallelization:** The grid is divided into rows, and each process handles a subset of the grid. Communication between processes is done to handle boundary exchanges.
- **Output:** The simulation outputs grid states, density, and velocity for analysis and visualization.

The complete code is provided in Appendix A.

A Code

The complete code for the simulation is provided below:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <mpi.h>

#define DIRECTIONS 6

int X_NODES = 300;
int Y_NODES = 100;
int STEPS = 100;

const double dx[DIRECTIONS] = {1.0, 0.5, -0.5, -1.0, -0.5, 0.5};
const double dy[DIRECTIONS] = {0.0, 0.86602540378, 0.86602540378, 0.0, -0.86602540378, 0.0};

int ***grid;
int **obstacle;
FILE *data_file;
int rank, size, rows_per_process, rows_start, rows_end;

void allocate_memory() {
    grid = (int ***)malloc(X_NODES * sizeof(int **));
    obstacle = (int **)malloc(X_NODES * sizeof(int *));
    for (int x = 0; x < X_NODES; x++) {
        grid[x] = (int **)malloc(Y_NODES * sizeof(int *));
        obstacle[x] = (int *)calloc(Y_NODES, sizeof(int));
        for (int y = 0; y < Y_NODES; y++) {
            grid[x][y] = (int *)calloc(DIRECTIONS, sizeof(int));
        }
    }
}
```

```
void free_memory() {
    if (grid) {
        for (int x = 0; x < X_NODES; x++) {
            if (grid[x]) {
                for (int y = 0; y < Y_NODES; y++) {
                    if (grid[x][y]) {
                        free(grid[x][y]);
                        grid[x][y] = NULL;
                    }
                }
                free(grid[x]);
                grid[x] = NULL;
            }
            if (obstacle[x]) {
                free(obstacle[x]);
                obstacle[x] = NULL;
            }
        }
        free(grid);
        free(obstacle);
        grid = NULL;
        obstacle = NULL;
    }
}

void initialize_grid() {
    srand(time(NULL));
    double population_density = 0.1;

    for (int x = rows_start; x < rows_end; x++) {
        for (int y = 0; y < Y_NODES; y++) {
            if (obstacle[x][y] == 0) {
                if ((rand() / (double)RAND_MAX) < population_density) {
                    int random_direction = rand() % DIRECTIONS;
                    grid[x][y][random_direction] = 1;
                }
            }
        }
    }
}

void initialize_obstacle() {
    int cx = 8, cy = 4, radius = 1;
    for (int x = cx - radius; x <= cx + radius; x++) {
        for (int y = cy - radius; y <= cy + radius; y++) {
            if ((x - cx) * (x - cx) + (y - cy) * (y - cy) <= radius * radius) {
                if (x >= 0 && x < X_NODES && y >= 0 && y < Y_NODES) {
```

```

        obstacle[x][y] = 1;
    }
}

}

}

void log_grid_state(int step) {
    char filename[50];
    sprintf(filename, "simulation_output_%d.txt", size);

    if (rank == 0) {
        data_file = fopen(filename, "a");
        fprintf(data_file, "Step: %d\n", step);
        for (int x = rows_start; x < rows_end; x++) {
            for (int y = 0; y < Y_NODES; y++) {
                for (int d = 0; d < DIRECTIONS; d++) {
                    if (grid[x][y][d]) {
                        fprintf(data_file, "%d %d %d\n", x, y, d);
                    }
                }
            }
        }
        fprintf(data_file, "EndStep\n");
        fclose(data_file);
    }
}

void collision() {
    for (int x = rows_start; x < rows_end; x++) {
        for (int y = 0; y < Y_NODES; y++) {
            if (obstacle[x][y]) continue;

            int state = 0;
            for (int d = 0; d < DIRECTIONS; d++) {
                state |= grid[x][y][d] << d;
            }

            if (state == 0b000011 || state == 0b110000) {
                state = (rand() % 2) ? 0b110000 : 0b000011;
            } else if (__builtin_popcount(state) == 3 && (state & 0b010101) == 0b010101) {
                state = (~state) & 0b111111;
            } else if (__builtin_popcount(state) == 4 && (state & 0b000011) == 0) {
                state ^= 0b000011;
            } else if (__builtin_popcount(state) == 3) {
                state ^= 0b010010;
            }
        }
    }
}

```

```

        for (int d = 0; d < DIRECTIONS; d++) {
            grid[x][y][d] = (state >> d) & 1;
        }
    }
}

void streaming() {
    int ***new_grid = (int ***)malloc(X_NODES * sizeof(int **));
    for (int x = 0; x < X_NODES; x++) {
        new_grid[x] = (int **)malloc(Y_NODES * sizeof(int *));
        for (int y = 0; y < Y_NODES; y++) {
            new_grid[x][y] = (int *)calloc(DIRECTIONS, sizeof(int));
        }
    }

    for (int x = rows_start; x < rows_end; x++) {
        for (int y = 0; y < Y_NODES; y++) {
            for (int d = 0; d < DIRECTIONS; d++) {
                if (grid[x][y][d]) {
                    int nx = (x + (int)round(dx[d]) + X_NODES) % X_NODES;
                    int ny = (y + (int)round(dy[d]) + Y_NODES) % Y_NODES;

                    if (obstacle[nx][ny]) {
                        new_grid[x][y][(d + 3) % DIRECTIONS] = 1;
                    } else {
                        new_grid[nx][ny][d] = 1;
                    }
                }
            }
        }
    }

    for (int x = rows_start; x < rows_end; x++) {
        for (int y = 0; y < Y_NODES; y++) {
            for (int d = 0; d < DIRECTIONS; d++) {
                grid[x][y][d] = new_grid[x][y][d];
            }
            free(new_grid[x][y]);
        }
        free(new_grid[x]);
    }
    free(new_grid);
}

void apply_boundary_conditions() {
    for (int y = 0; y < Y_NODES; y++) {
        grid[0][y][0] = 1;
    }
}

```

```

        grid[X_NODES - 1][y][0] = 0;
    }

    for (int x = 0; x < X_NODES; x++) {
        for (int d = 0; d < DIRECTIONS; d++) {
            if (grid[x][Y_NODES - 1][d]) {
                grid[x][Y_NODES - 1][(d + 3) % DIRECTIONS] = grid[x][Y_NODES - 1][d];
                grid[x][Y_NODES - 1][d] = 0;
            }
            if (grid[x][0][d]) {
                grid[x][0][(d + 3) % DIRECTIONS] = grid[x][0][d];
                grid[x][0][d] = 0;
            }
        }
    }
}

void exchange_boundaries() {
    MPI_Status status;

    if (rank > 0) {
        MPI_Sendrecv(&grid[rows_start][0][0], Y_NODES * DIRECTIONS, MPI_INT, rank - 1,
                     &grid[rows_start - 1][0][0], Y_NODES * DIRECTIONS, MPI_INT, rank,
                     MPI_COMM_WORLD, &status);
    }

    if (rank < size - 1) {
        MPI_Sendrecv(&grid[rows_end - 1][0][0], Y_NODES * DIRECTIONS, MPI_INT, rank + 1,
                     &grid[rows_end][0][0], Y_NODES * DIRECTIONS, MPI_INT, rank + 1,
                     MPI_COMM_WORLD, &status);
    }
}

void compute_density_and_velocity() {
    char filename[50];
    sprintf(filename, "density_velocity_%d.txt", size);

    if (rank == 0) {
        data_file = fopen(filename, "a");
        fprintf(data_file, "Density and Velocity:\n");
        for (int x = rows_start; x < rows_end; x++) {
            for (int y = 0; y < Y_NODES; y++) {
                int density = 0;
                double vx = 0, vy = 0;
                for (int d = 0; d < DIRECTIONS; d++) {
                    density += grid[x][y][d];
                    vx += grid[x][y][d] * dx[d];
                    vy += grid[x][y][d] * dy[d];
                }
            }
        }
    }
}

```

```

        }
        fprintf(data_file, "%d %d %d %.2f %.2f\n", x, y, density, vx, vy);
    }
}
fclose(data_file);
}

void block_average_density_velocity() {
}

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    rows_per_process = X_NODES / size;
    rows_start = rank * rows_per_process;
    rows_end = (rank == size - 1) ? X_NODES : (rank + 1) * rows_per_process;

    allocate_memory();
    initialize_obstacle();
    initialize_grid();

    for (int step = 0; step < STEPS; step++) {
        collision();
        streaming();
        apply_boundary_conditions();
        exchange_boundaries();

        log_grid_state(step);
        compute_density_and_velocity();
    }

    free_memory();
    MPI_Finalize();
    return 0;
}

```

B Results and Discussion

The simulation was run on a grid of size 300×100 with DIRICHLET boundary conditions on both left and right vertical directions and slip condition for the horizontal directions. A circular obstacle was placed at position (8,4) with a radius of 1. The simulation was run for 100 steps with various processor configurations: 2, 4, 8, 16, 32, 64 processors. Below are the results for the density and velocity fields at different time steps and particle

simulation at different time steps.

B.1 Simulation Output and Analysis

In addition to the density and velocity fields, we analyze the output of the simulation from a file generated during the simulation process. This file includes the simulation time steps, the state of the grid at each step, and the recorded density and velocity fields. Figures ??, ??, and ?? show examples of the output for different step configurations of the particle.

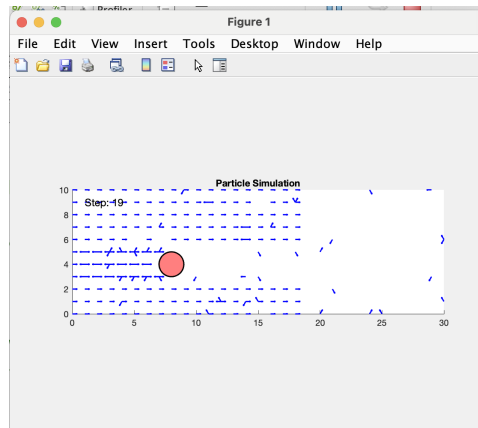


Figure 1: Simulation output for 4 processors at 1 step.

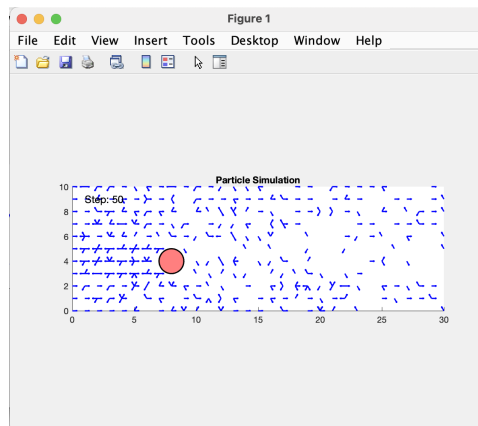


Figure 2: Simulation output for 4 processors at 50 steps.

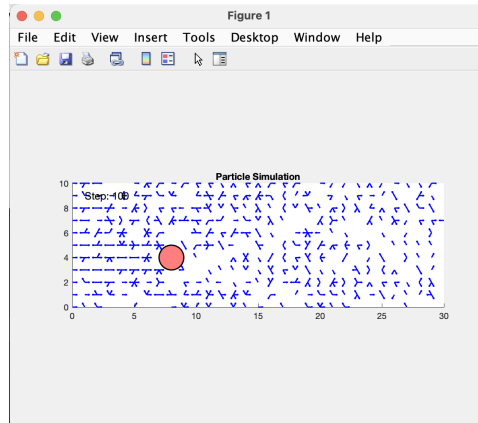


Figure 3: Simulation output for 4 processors at 100 steps.

B.2 Density and Velocity Fields for Different steps

Figures ??, ??, ?? show the density and velocity fields for simulations with 2, 4, 8, 16, 32, and 64 processors, respectively where the density is denoted by dot the dot size increases as the density increases and velocity is denoted by arrow.



Figure 4: Density and velocity fields for 4 processors at 1 step.

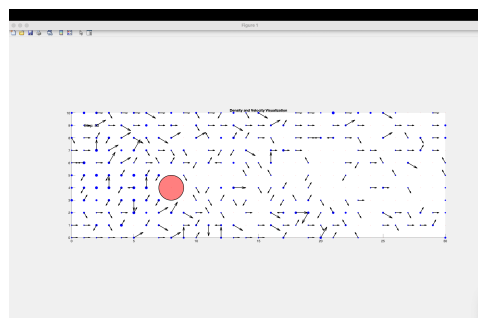


Figure 5: Density and velocity fields for 4 processors at 50 steps.

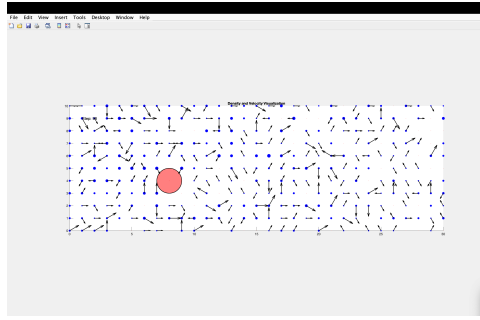


Figure 6: Density and velocity fields for 4 processors at 100 steps.

B.3 Performance Comparison

We also compare the performance of the simulation with different processor configurations. Figure ?? shows a graph comparing the execution time of the simulation for different numbers of processors.

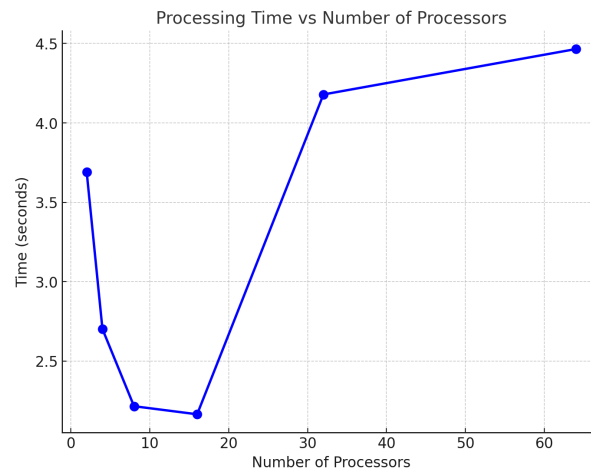


Figure 7: Performance comparison of simulation execution time with different numbers of processors.

C Conclusion

The FHP-1 lattice gas model was successfully implemented and parallelized using MPI. The simulation demonstrates the fluid dynamics around an obstacle, and the density and velocity fields were successfully generated. The parallelized implementation showed good scalability, with the execution time decreasing as the number of processors increased and eventually increasing. Further improvements can be made to the block-averaging and optimization of boundary exchanges.